

The Situation

AIE 190x is a project-based course with multiple sessions, each session has a different theme. Each session has a strict capacity, say, 30 seats.

Before the start of official enrollment period, students are given the description of the available sessions. Students are asked to submit their ranked preferences (1st choice, 2nd choice, etc.) before a deadline.

After the deadline has passed, and you must assign every student to a session.

The Problem

You cannot just assign students in the order they submitted their preferences—that would unfairly reward early birds. Instead, the teaching team wants the **overall happiest group of students possible**, minimizing the total "disappointment score" (1 point for a 1st choice, 2 points for a 2nd choice, etc.). Students who cannot be placed in any of their preferred sessions are "unassigned" and receive a high penalty.

However, the teaching team also acknowledges that searching for the *perfect* assignment might take forever. So, you are allowed to use a **randomized approach**:

- Run many random trials (e.g., 100).
- In each trial, randomly shuffle the student list, then assign them one by one using the "serial dictatorship" rule (each student gets their highest-ranked available session).
- Track the best few assignments you find.
- Stop early if you stop seeing improvements.

The teaching team demands **reproducibility**—the same run should produce the same result every time (so you must use a fixed random seed).

Your Task

Write a program that:

1. **Reads** two CSV files:

- `students.csv` (student ID, name, and ranked preferences in columns)
- `sessions.csv` (session name and capacity)

2. **Runs the randomized algorithm** with configurable parameters:

- Random seed
- Number of trials (e.g., 100)
- Number of top assignments to track (e.g., 5)
- Early stopping patience (stop after, say, 20 trials with no improvement)

3. **Finds the best assignment** (lowest total disappointment score).

4. **Outputs:**

- `assignment.csv` – the best assignment, showing each student's assigned session and their rank.
- `report.txt` – a summary including total cost, how many students got 1st/2nd/3rd choices, session utilization, and the top assignments found.

The Rules (Technical Specifications)

- **Cost system:** 1st choice = 1 point, 2nd choice = 2 points, etc. Unassigned students = penalty of (number of sessions + 1) points.
- **No session may exceed its capacity.**
- **Random shuffling** must use the provided seed so results are reproducible.
- **Track the top N assignments** (lowest total cost) across all trials.
- **Early stopping:** Stop if no improvement for P consecutive trials, or if you hit the theoretical minimum (all students get 1st choice).
- **Standard libraries only.** No external optimization or machine learning packages.

What You Turn In

- All source code
- A `README.md` explaining how to run your program and any design decisions
- Sample output for the small test data you create